

A Comprehensive analysis of the CBP 2025 Branch Predictors entries: LVCP, RVA-Toru, and TAGE-SC 2025

Saumya Patel

March 28, 2026

Contents

1	Introduction	1
1.1	The Oracle Predictor	1
2	Methodology	1
3	MultiPerspective Perceptron Predictor (MPP)	2
3.1	Details	2
3.1.1	Branch Misalignment and MODHIST	2
3.2	MPP vs Baseline Predictor	3
4	Branch Prediction via Load Value Prediction (BALL)	4
4.1	Details	4
4.2	BALL Predictor vs Baseline	6
5	Bullseye Predictor	7
5.1	Details	7
5.2	Bullseye Predictor vs Baseline	8
6	Programming Idiom Predictor (PIP)	10
6.1	Details	10
6.2	Programming Idiom Predictor vs Baseline	11
7	Delta Analysis	12
7.1	Bullseye vs Multi-Perspective Predictor	12
7.2	BALL vs Bullseye Predictor	14
8	Complementarity Analysis	15

1 Introduction

In the previous assignment, I evaluated the following branch predictors:

- Load Value Correlator Predictor (LVCP)
- Register-Value-Aware Branch Predictor (RVA-Toru)
- TAGE-SC 2025 André Seznec Predictor
- TAGE-SC-L Alberto Ros Predictor

In this report, I will continue my analysis of these predictors by performing a similar analysis of the remaining CBP2025 entries, namely

- MultiPerspective Perceptron Predictor (MPP) by Jimenez, et al.
- Programming Idiom Predictor (PIP)
- Branch Prediction via Load Value Prediction (BALL) by Jun Fan
- The Bullseye Predictor by Emet Behrendt, et al.

I will do a similar analysis of these predictors as I did for the first four and then I will compare all 8 predictors together to draw a final conclusion and rank them based on their performance.

1.1 The Oracle Predictor

The Oracle predictor is a theoretical construct that represents the best possible branch predictor that is not ahead of time. The point of this predictor is to check the maximum possible performance of a branch predictor on the given traces. It is important to note that the Oracle predictor is not a real predictor and cannot be implemented in hardware, but it serves as a useful benchmark for evaluating the performance of other predictors. More implementation details about the Oracle predictor can be found in the later sections of this report.

2 Methodology

The methodology for evaluating the predictor is the same as the one used in the previous assignment. I will be using the CBP2025 traces to evaluate the performance of the predictors. I will be using the same metrics as before, namely

- Branches Per Cycles (BrPerCyc)
- Misprediction Branches Per Cycles (MispBrPerCyc)
- Misprediction Rate (MR)
- Misprediction Rate Per Kilo Instructions (MPKI)
- Wrong Path Cycles (CycWP)
- Wrong Path Cycles Per Kilo Instructions (CycWPKI)

Furthermore, I also compared the predictors against each other to draw a final conclusion about their performance. The metrics that I used were,

- **Delta Misprediction Per Kilo Instructions:** $\Delta_{MPKI} = MPKI_A - MPKI_B$
where A and B are two predictors being compared. This metric gives us a direct comparison of the misprediction rates of the two predictors.

3 MultiPerspective Perceptron Predictor (MPP)

3.1 Details

The Paper argues that traditional predictors look at branch history from two angles, the global and the local history. Jimenez argues that there are many useful "perspectives" on the same history. He introduces the MPP which is essentially a hashed perceptron that combines many of these perspectives together. The new features that this predictor introduces are the following:

- **MODHIST (Modulo History):** The author describes that he noticed a problem that he called *branch misalignment* i.e. if branch B sometimes appears in the history and some does not, then the predictor will have a hard time learning the pattern, because the same outcome sequence would land at different positions in the history register. This problem has seemed to break traditional and hashed predictors. To solve this problem, the author introduces MODHIST which essentially only records branches whose addresses are divisible by some modulus, filtering out the "misaligned" branches and thus making the history more consistent.
- **RECENCYPOS:** It tracks a stack of recently executed branches using LRU policy. knowing where the branch is in this stack can be useful to correlate the outcome of the branch with its recency.
- **BLURRYPATH:** It records coarser memory regions instead of exact addresses. A "coarser" memory region is defined as regions of memory who have the first N bits of their address the same. For example, if you take the address `0x7FFF4A28` and right shift it by 4 bits you would get `0x7FFF4A`, which would mean that every address that only differs in those last 4 bits would be considered the same "blurry" address.

You would want to track these "blurry" addresses because of 2 reasons, **first**, they have less noise. If your program takes a slightly different path through the same general code region, precise history says "completely different." Blurry history says "same region, close enough." This can actually generalize better because you're capturing the structural shape of execution rather than its exact fingerprint. **Second**, compression. Fewer distinct values means less aliasing pressure in the hash tables which means that different histories are less likely to collide on the same table entry.

MPP goes one step further, it only shifts a new region into the history when you cross into a new region. So if five consecutive branches are all within the same 256-byte chunk, only one entry gets recorded. You're tracking region transitions, not individual branches. This naturally compresses long sequences of nearby branches into a compact history

- **ACYCLIC history:** This tries to get the repetitive loop structure from the history, keeping only the most recent outcome of each branch address rather than the full sequence of outcomes. This is useful because in many cases, the most recent outcome of a branch is more predictive than the full history, especially in loops where the same branches are executed multiple times.
- **Forward Branch IMLI:** Usually IMLI is designed for backward branches, but the author has added a version for *top testing* loops i.e. where the exit is at the top and the unconditional jump is at the bottom. This is a very common optimization that compilers do (check [here](#) for reference).

3.1.1 Branch Misalignment and MODHIST

Consider the following code snippet:

```

1 if (x > 0) {
2     if (x < 100) {          % the potentially misaligning branch
3         do_something();
4     }
5 }

```

When $x > 0$ and $x < 100$, both branches execute and both get recorded into the history register. When $x > 0$ but $x \geq 100$, only the outer branch executes. This means the inner branch sometimes shows up in history and sometimes does not, which causes a problem.

Suppose there are two other branches later in the code, call them B and C, that always execute after this block. When the inner branch ran, B and C get pushed one slot further back in the history register. When it did not run, B and C sit one slot closer to the front. Same program, same behavior from B and C, but they appear at different positions in history depending on whether that inner branch fired.

TAGE hashes based on exact bit positions, so it sees these two situations as completely different and stores them in different table entries. Each entry gets trained half as often and the predictor never builds enough confidence to predict reliably. This is the misalignment problem.

MODHIST fixes this by simply not recording branches whose address does not satisfy:

`branch_address mod N == 0`

The idea is that guard branches like null checks and bounds checks, which are the typical culprits behind misalignment, tend to land at non-aligned addresses in compiled code. By filtering them out, B and C always appear at the same position in history, and the predictor can finally learn a consistent pattern.

3.2 MPP vs Baseline Predictor

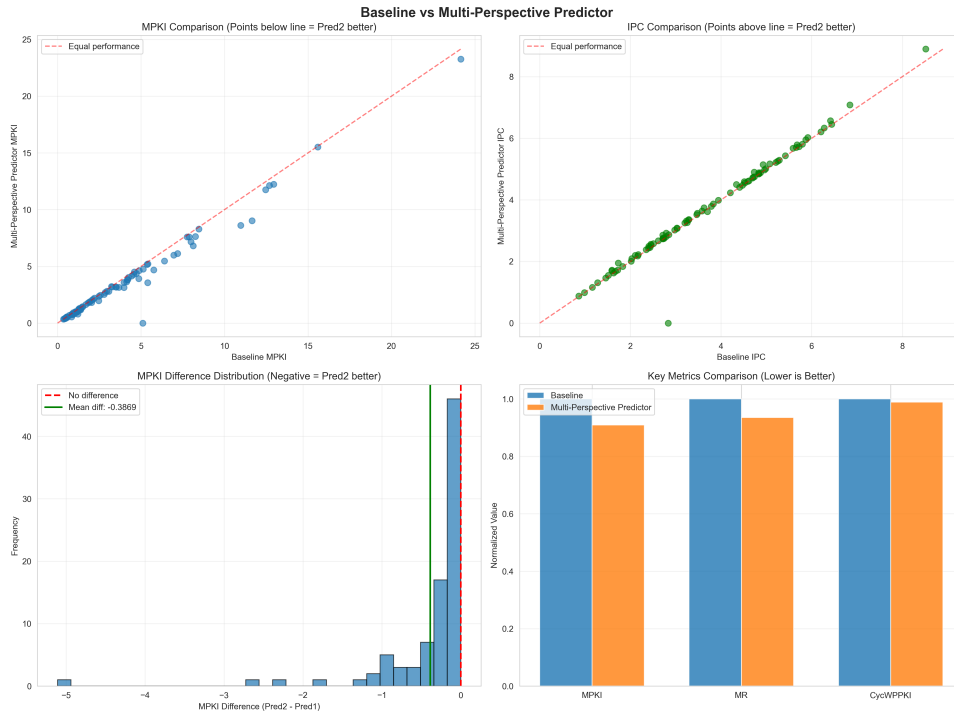


Figure 1: Comparison of MPP and Baseline Predictors

MPP proves to perform better than the baseline predictor, achieving a mean MPKI difference of 0.38 across all traces. This is a pretty decent improvement. One of the most interesting things

about the MPP is that it uses two bloom filters to identify branches that are trivially T or NT. Looking at the top 10 improvements, 6 of them are *int_* traces. As mentioned before, integer code is compiled from high level languages and has complex control flow, and compilers usually generate *alignment breaking branches* as cited in the paper,

“A branch branches over another branch. The second branch sometimes appears in the branch history and sometimes does not appear, leading to the same branch outcomes appearing in different locations in the global history.”

These branches appear as null checks, type checks and bound checks. This is where TAGE’s strength becomes an issue i.e. a single misaligned branch “poisons” a long history entry. The MODHIST as mentioned above aims to solve this particular problem, and it seems to be doing a good job at it.

Table 1: Framework-Level Summary: MPP vs Baseline Predictor

Framework	Improve/Total	Success Ratio (%)	Total MPKI Improvement
Web	26/26	100.0	-10.4414
Int	28/37	75.7	-18.8957
Fp	8/14	57.1	-2.3088
Media	3/4	75.0	-0.5673
Infra	5/16	31.3	-1.3662
Compress	1/8	12.5	-0.9672

The improvement in *Web_* traces is not as dramatic but is remarkably consistent, having a 100% success rate. The precise path history of web traces becomes too specific, i.e. two slightly different web page loads produce completely different path histories even though their branch behaviour is very similar. The MPP’s use of the BLURRYPATH feature allows it to generalize better and thus achieve a consistent improvement across all web traces.

The compress and infra traces are interesting because they show very little improvement over the baseline. For the compress traces I think the explanation is fairly simple. Their baseline MPKI is already very low, so TAGE-SC-L is already doing a good job and there is not much left for MPP to improve. The infra traces are honestly less clear to me. My best guess is that these workloads contain branches whose outcomes depend more on runtime data values than on control flow history, in which case none of MPP’s history-based features would help. It is also possible that MODHIST’s filtering is actually discarding branches that carry useful signal in these specific workloads, which would work against MPP rather than helping it. That said, I want to be clear that these are speculative explanations. Without knowing exactly what workloads these traces represent, it is difficult to say with confidence why MPP struggles to improve them.

4 Branch Prediction via Load Value Prediction (BALL)

4.1 Details

The BALL (Branch-ALU-Load-Load) predictor is motivated by a key limitation of history-based predictors like TAGE-SC-L: they struggle with **load-data-dependent branches**. These are branches whose outcomes depend on values loaded from memory, rather than patterns in branch history. For such branches, the issue is not insufficient history or storage, but rather that the branch outcome is determined by a chain of computations involving loads and ALU operations. Since TAGE does not explicitly model this data dependency, it fails to capture these patterns

effectively. The BALL predictor addresses this by explicitly reconstructing the **data dependence chain** leading up to a mispredicted branch. Instead of predicting the branch directly, it predicts the intermediate values (loads and ALU results) and uses them to compute the final branch outcome. The predictor is designed as a **complementary subsystem** to TAGE-SC-L. When TAGE mispredicts a branch, BALL attempts to handle it using a different strategy based on value prediction. The BALL predictor consists of the following main components:

- **Dependence Chain Construction:** When a branch is mispredicted, BALL constructs a limited **Branch-ALU-Load-Load** chain by tracking RAW (read-after-write) dependencies in recently committed instructions. This chain captures how the branch depends on one ALU operation and up to four load instructions. The key idea is to isolate only the critical instructions that directly influence the branch outcome.
- **Load Address and Value Prediction:** Instead of predicting load values directly, BALL first predicts **load addresses**, and then reads a specialized data structure (BP Data Cache) to obtain the predicted values. Load addresses are predicted using three main patterns:
 - Constant stride (fixed or incremental patterns)
 - Pointer chasing (using previous load values as base addresses)
 - Branch-path history (via a modified E-VTAGE predictor)

This two-step process allows BALL to recover the actual data values that influence the branch.

- **ALU and Flag Prediction:** Once load values are predicted, BALL simulates the ALU operation (typically comparisons) to compute the **flag register value**. These operations are simple (e.g., compare two values or compare against zero) and are learned and stored using small pattern tables indexed by PC.
- **Branch Outcome Prediction and Selection:** The predicted flag value is then used to determine the final branch direction. BALL only makes a prediction when all required intermediate values are available with high confidence. A selector counter arbitrates between BALL and TAGE-SC-L. If BALL consistently predicts correctly for a branch, it is given priority; otherwise, TAGE remains the default predictor.

Overall, BALL shifts the prediction problem from **control-flow correlation** (used by TAGE) to **data-flow reconstruction**. By explicitly modeling how data values influence branches, it is able to accurately predict cases that are inherently difficult for traditional predictors.

4.2 BALL Predictor vs Baseline

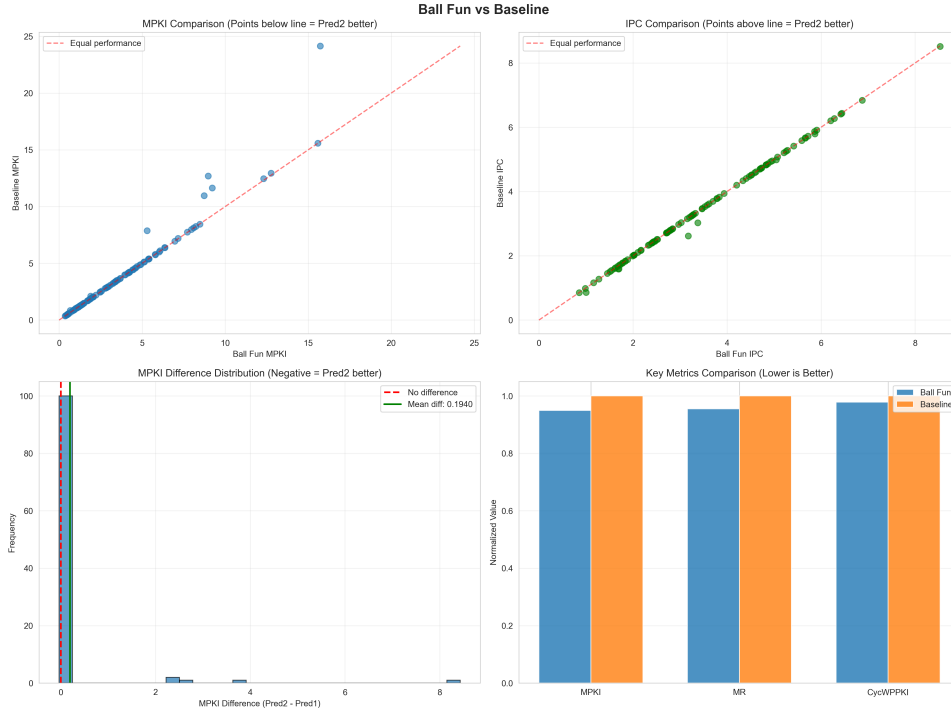


Figure 2: Comparison of BALL and Baseline Predictors

The BALL predictor is an unusual case in this comparison. Unlike MPP and Bullseye which are designed to be general-purpose improvements over TAGE-SC-L, BALL was never intended to replace it. It was designed to work alongside TAGE-SC-L as a complementary predictor, specifically targeting branches whose outcomes depend on values loaded from memory. Running it as a standalone predictor against the baseline is therefore testing it in a way it was never designed for, and the graphs show this clearly.

Looking at the MPKI scatter plot, almost every single point sits directly on the equal performance line. Almost all traces show no difference compared to the baseline predictor. The histogram makes this even clearer. Nearly all 105 traces land in a single bin right at zero, with a mean difference of only 0.194 MPKI. For the vast majority of workloads, BALL and the baseline are producing the same predictions.

The only reason BALL shows any improvement at all is because of a very small number of traces that are different from this pattern. As shown in Table 2, there are exactly 9 traces where BALL makes a real difference, and these are the isolated points visible pulling away from the diagonal in the scatter plot.

The biggest result here is `int_21`, which drops by 8.43 MPKI. This is the largest improvement of any predictor on any single trace in this entire comparison. The `fp_13` and `fp_8` traces show a similar pattern. These are not random. These are the exact traces used as examples in the BALL paper itself. They contain branches that sit at the end of load dependency chains with predictable address patterns, which is exactly the problem BALL was built to solve.

For everything else, BALL has nothing to offer. It does not activate on compress, infra, media, or web workloads because those branches are not load-data-dependent in the way BALL needs them to be. The normalized metrics bar chart confirms this. BALL is better across MPKI, MR and CycWPPKI overall, but only slightly, because those gains come from just 9 traces spread across 105 total.

In short, BALL is a very specialised predictor. It solves one specific problem very well and does nothing outside of that problem. Whether that is useful depends entirely on what workloads

Table 2: The Only Traces Where BALL Made a Meaningful Difference

Trace	Baseline MPKI	BALL MPKI	Improvement
int_21	24.1502	15.7181	-8.4321
fp_13	12.6898	8.9772	-3.7126
fp_8	7.8738	5.2985	-2.5753
int_1	11.6458	9.2211	-2.4247
int_2	10.9652	8.7270	-2.2382
fp_10	2.1174	1.8923	-0.2251
int_19	0.8482	0.6642	-0.1840
int_30	12.9427	12.7675	-0.1752
int_29	12.4572	12.3072	-0.1500
All other traces (96)		No meaningful difference	

you care about.

5 Bullseye Predictor

5.1 Details

The bullseye predictor is driven by a core observation, stating that most of the mispredictions in TAGE-SC-L come from a small number of "hard-to-predict" branches. The bullseye predictor is designed to identify these hard-to-predict branches and apply a specialized prediction strategy to them, while leaving the rest of the branches to be handled by a more traditional predictor. H2P branches appear under different global histories every time they execute. For a predictor like TAGE that relies on global history, this is a very difficult pattern to learn. The paper also notes that simply making TAGE bigger doesn't fix this problem, which means even an infinitely large TAGE would barely improve the prediction rates. The main problem isn't storage, but the branch's behaviour not correlating well enough with the global history.

The bullseye predictor adds a small 256k "subsystem" on top of the TAGE-SC-L predictor. It is composed of 3 main components:

- **H2P (Hard-to-Predict) Branch Classifier:** This component is responsible for identifying the hard-to-predict branches. This tracks every branch's execution count, MP count and running accuracy (the number of correct predictions divided by the total number of predictions for that branch). When a branch's execution count exceeds a certain threshold and its accuracy falls below a certain number, the classifier flags the branch as an H2P branch, preventing the entire system from being overwhelmed by the noise of these branches and allowing it to focus on the branches that it can predict well. In practice no more than 10 branches are active at once.
- **Two Perceptron Predictors:** Bullseye predictor uses two perceptron predictors, both of them running in parallel with the TAGE-SC-L predictor. One of them uses local history and the other uses folded global history to essentially "catch" long-range correlation that TAGE-SC-L misses. These perceptrons dedicate their weights to one hard branch rather than sharing tables across and with thousands of branches (something that TAGE does).
- **Trial phase and arbitration:** A newly flagged branch gets 512 executions to warm up its weights. After that the system compares which predictor is winning (like a tournament predictor). There is a confidence counter that reinforces the winning predictor and allows it to take over the prediction of that branch. The TAGE is set to default, but as soon as the

perceptron wins 128 consecutive predictions, the TAGE updates are suppressed, stopping the thrashing cycle (where TAGE and the perceptron keep taking over the prediction of the branch and thus never giving either of them enough time to learn the pattern) once and for all.

5.2 Bullseye Predictor vs Baseline

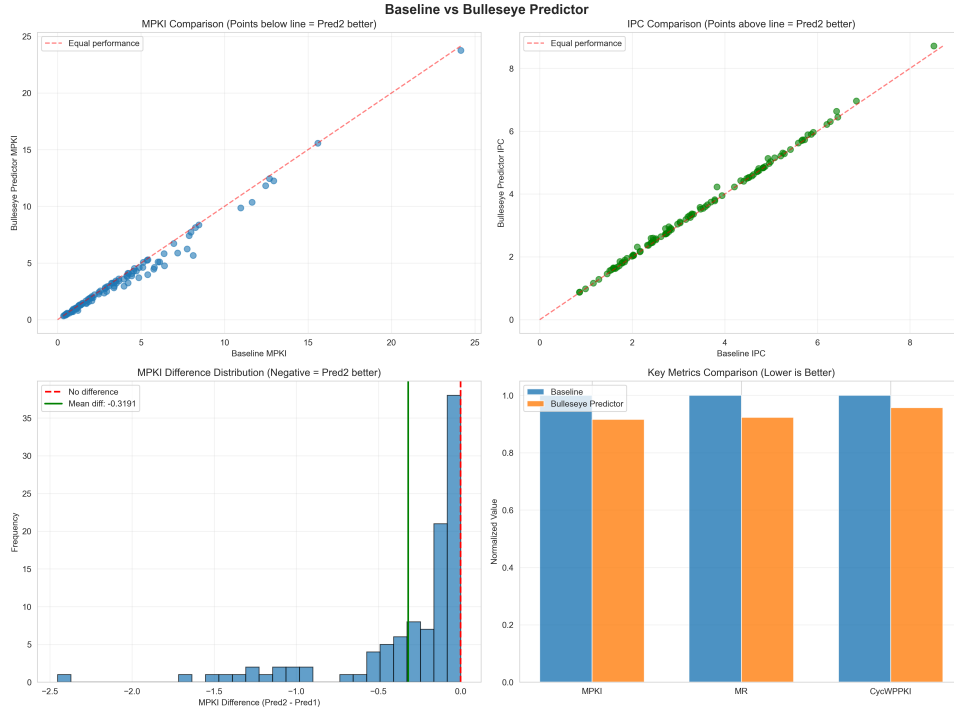


Figure 3: Comparison of Bullseye and Baseline Predictors

The Bullseye predictor is built around one core idea: most of the mispredictions in TAGE-SC-L come from a very small number of hard-to-predict (H2P) branches, so rather than trying to improve prediction across the board, it identifies those specific branches and handles them with dedicated perceptron predictors. The results here show that this strategy works well, with Bullseye winning on 104 out of 105 traces and achieving an overall MPKI of 3.489 compared to the baseline’s 3.808, which is an 8.38% improvement.

Table 3 shows the overall summary statistics across all key metrics.

Table 3: Overall Performance Summary: Baseline vs Bullseye

Metric	Baseline	Bullseye	Improvement
MPKI	3.8080	3.4889	8.38% better
MR	2.9679	2.7395	7.70% better
CycWPPKI	161.60	154.64	4.31% better
IPC	3.4636	3.5132	1.43% better
Cycles	14,824,660	14,593,783	1.56% better

Unlike BALL which only helped on 9 specific traces, Bullseye improves almost every single trace to some degree. The only regression in the entire dataset is `fp_4`, which gets worse by just 0.0013 MPKI, which is essentially nothing. This tells us that Bullseye’s trial phase and

confidence arbitration are doing their job well: the perceptron only takes over a branch when it has proven it can do better than TAGE-SC-L, so there is almost no risk of making things worse.

Table 4 breaks down the average MPKI by workload category, which shows where Bullseye helps most and where it has less impact.

Table 4: Average MPKI by Workload Category: Baseline vs Bullseye

Category	Baseline MPKI	Bullseye MPKI	Improvement
INT	4.3284	3.5831	17.22%
WEB	3.8812	3.4033	12.31%
FP	3.5456	3.3566	5.33%
COMPRESS	2.7957	2.7583	1.34%
INFRA	2.6335	2.5634	2.66%
MEDIA	1.0661	0.9954	6.63%
Overall	3.8080	3.4889	8.38%

The integer and web categories benefit the most. For integer workloads, the improvement is 17.22%, which makes sense because integer code tends to have complex control flow with many hard branches that consistently defeat TAGE-SC-L. These are exactly the kind of branches that Bullseye’s H2P Identification Table (HIT) flags and routes to a dedicated perceptron. The top individual improvements back this up, with `int_16` dropping by 2.45 MPKI, `int_17` by 1.64 MPKI, and `int_8`, `int_7` and `int_22` all dropping by over 1.0 MPKI each.

The web category also sees strong improvement at 12.31%. Web workloads jump across many different code regions, which produces branches that appear under very different global histories each time they run. This is the exact H2P profile that Bullseye is designed to catch, and `web_7` dropping by 1.51 MPKI is a good example of this.

Compress and infra remain largely unchanged, which is consistent with what we saw for the other predictors. These workloads have relatively low baseline MPKI to begin with, so the HIT thresholds are rarely crossed and Bullseye simply never activates for most of these branches. There is nothing for it to fix.

Table 5 shows the top traces where Bullseye made the biggest difference.

Table 5: Top Traces by MPKI Improvement: Baseline vs Bullseye

Trace	Baseline MPKI	Bullseye MPKI	Improvement
<code>int_16</code>	8.1253	5.6722	-2.4531
<code>int_17</code>	6.3949	4.7592	-1.6357
<code>web_7</code>	7.7526	6.2471	-1.5055
<code>int_22</code>	5.3965	3.9788	-1.4177
<code>int_8</code>	7.1931	5.8786	-1.3145
<code>int_7</code>	5.7589	4.4556	-1.3033
<code>int_1</code>	11.6458	10.3681	-1.2777
<code>web_11</code>	5.7895	4.6089	-1.1806
<code>int_9</code>	4.8555	3.7132	-1.1423
<code>int_2</code>	10.9652	9.8611	-1.1041
Only regression			
<code>fp_4</code>	0.5736	0.5749	+0.0013

One thing worth noting is that `int_21`, which has the highest baseline MPKI of any trace at 24.15, only improves by 0.39 MPKI under Bullseye. This is an interesting contrast with BALL,

which reduces `int_21` by 8.43 MPKI. The reason is that `int_21`'s hard branches are load-data-dependent, which is a problem that perceptron predictors cannot solve through history patterns alone. Bullseye identifies the branch as H2P but its perceptrons cannot find a reliable history-based pattern for it either, so the improvement is minimal. This highlights the key difference between Bullseye and BALL: Bullseye is broadly useful across many types of hard branches, while BALL solves one specific type of hard branch much more deeply.

6 Programming Idiom Predictor (PIP)

6.1 Details

The PIP (Programming-Idiom Predictor) is based on a key idea: a large portion of branches in real programs follow common programming patterns (idioms), and instead of trying to learn these patterns indirectly through history, it is more effective to detect them explicitly and predict them directly. Rather than relying solely on history-based predictors like TAGE, PIP introduces a set of small, specialised predictors called idiom trackers, each designed to recognize a specific type of branch behaviour. If none of these trackers can make a prediction, the system falls back to a strong baseline predictor composed of TAGE-SC-L and a Multiperspective Perceptron (MPP).

The predictor is composed of 4 main components:

- **Idiom Trackers:** These are small, highly targeted predictors that detect specific programming patterns by observing execution behaviour such as instruction operands, memory accesses, and control flow. Two main trackers are used in this design:
 - **For-loop Tracker:** This tracker identifies loops of the form `for (i = B; i != E; i += S)` by observing arithmetic instructions whose differences decrease monotonically. Once detected, it predicts loop continuation or exit by projecting the current iteration count forward. Unlike traditional predictors, it can often predict the loop exit correctly on the very first encounter, sometimes achieving near 100% accuracy.
 - **Null-Terminated String (NTS) Tracker:** This tracker identifies loops iterating over strings (e.g., `*p != '\0'`) by observing memory loads with sequential addresses. It learns the length of strings and uses this information to predict when the loop will terminate in future executions.

Each tracker maintains confidence counters, and if its predictions are not useful, it is disabled to avoid polluting the system.

- **TAGE-SC-L Predictor:** This acts as the primary fallback predictor and is responsible for handling the majority of branches. It is a highly optimized version with increased history length, larger tables, and improved statistical correction mechanisms. It provides strong baseline accuracy for branches that follow history-based patterns.
- **Multiperspective Perceptron (MPP):** This predictor complements TAGE by using a wide range of features and correlations that TAGE may not capture. It is only relied upon when TAGE struggles, and it uses a modified training strategy that increases learning pressure when it disagrees with TAGE. This allows it to specialize in difficult branches without interfering with TAGE's strengths.
- **Arbiter:** When TAGE and the Perceptron disagree, the arbiter decides which predictor to trust. It uses confidence levels from TAGE and compares them against the magnitude of the perceptron's prediction. If TAGE is highly confident, it is chosen directly. Otherwise, the arbiter dynamically adjusts thresholds to determine when the perceptron should take over, learning over time which predictor performs better for different branches.

The overall prediction flow is straightforward: idiom trackers get the first chance to predict, and if none of them apply, the system falls back to TAGE and the perceptron, with the arbiter resolving conflicts. This design ensures that simple, structured patterns are handled perfectly by specialised hardware, while more complex or irregular branches are handled by powerful general-purpose predictors.

6.2 Programming Idiom Predictor vs Baseline

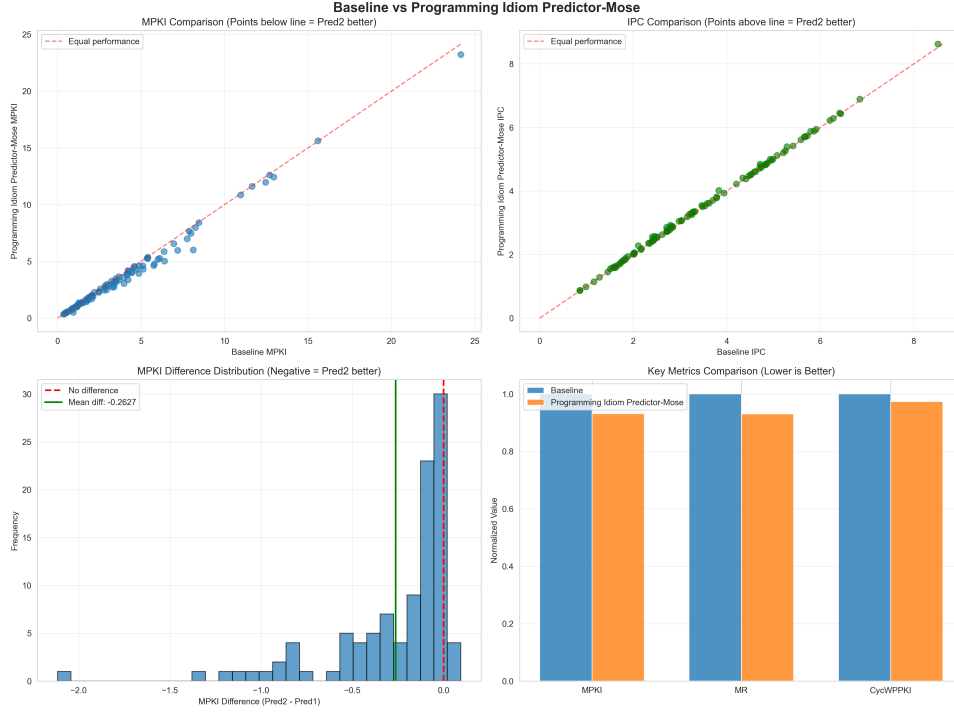


Figure 4: Comparison of Programming Idiom Predictor and Baseline Predictors

The results show a solid overall improvement, with the predictor winning on 97 out of 105 traces and achieving an overall MPKI of 3.545 compared to the baseline’s 3.808, which is a 6.90% improvement.

Table 6 shows the overall summary statistics across all key metrics.

Table 6: Overall Performance Summary: Baseline vs Programming Idiom Predictor

Metric	Baseline	Programming Idiom	Improvement
MPKI	3.8080	3.5453	6.90% better
MR	2.9679	2.7608	6.98% better
CycWPPKI	161.60	157.27	2.68% better
IPC	3.4636	3.4974	0.98% better
Cycles	14,824,660	14,667,399	1.06% better

The predictor improves broadly across most traces. It does have 8 regressions compared to Bullseye’s 1, but most of them are very small. The worst regression is `infra_8` which gets worse by only 0.094 MPKI, and the next worst is `infra_5` at 0.060 MPKI. These are small enough that they do not raise serious concerns.

Table 7 shows the average MPKI per workload category.

Table 7: Average MPKI by Workload Category: Baseline vs Programming Idiom Predictor

Category	Baseline MPKI	Programming Idiom MPKI	Improvement
WEB	3.8812	3.4657	10.70%
COMPRESS	2.7957	2.6599	4.86%
MEDIA	1.0661	1.0099	5.27%
INFRA	2.6335	2.5878	1.73%
INT	4.3284	4.3374	$\approx 0\%$
FP	3.5456	3.9243	$\approx 0\%$
Overall	3.8080	3.5453	6.90%

The web category sees the biggest improvement at 10.70%, which makes sense given that web workloads are full of branches tied to common programming patterns in parsers, string handlers and network code. The compress and media categories also see decent gains. The integer and fp category averages look flat or slightly worse, but this is a bit misleading. Looking at the individual trace results, the predictor actually wins on most integer and fp traces. The category average is being pulled down by a few very high MPKI traces like `fp_1` at 15.58 MPKI where the predictor simply cannot help. The regressions on `infra_8` and `infra_5` are the only two traces that stand out as real losses, and without knowing the exact nature of these workloads it is difficult to say why they regress.

Overall, the Programming Idiom Predictor sits between Bullseye and BALL in terms of its character. It is more broadly useful than BALL, improving traces across many categories rather than just a handful. It is slightly less consistent than Bullseye which had only one regression, but its overall MPKI improvement of 6.90% is competitive with Bullseye’s 8.38%.

7 Delta Analysis

7.1 Bullseye vs Multi-Perspective Predictor

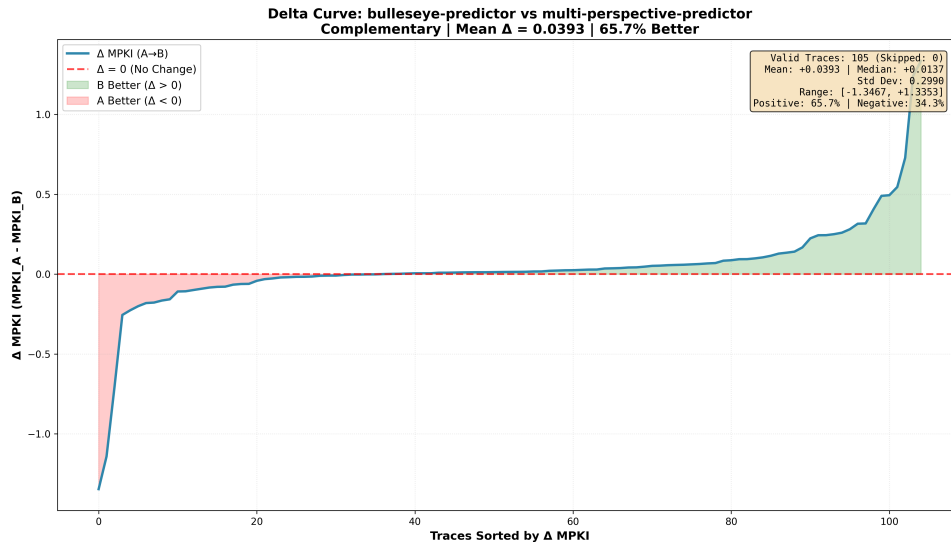


Figure 5: Delta Curve: Bullseye vs Multi-Perspective Predictor

Looking at the delta curve, the relationship between Bullseye and MPP is genuinely competitive. The curve crosses zero fairly early, meaning neither predictor dominates the other cleanly across all traces. Bullseye wins on 34.3% of traces (the pink region on the left) and MPP wins on 65.7%

of traces (the green region on the right), with a mean delta of +0.039 MPKI in favour of MPP overall.

Table 8 summarises the key numbers.

Table 8: Head to Head Summary: Bullseye vs MPP

Metric	Bullseye	MPP
Avg MPKI	3.4889	3.4495
Avg Miss Rate (%)	2.7395	2.7030
CycWPPKI	154.64	153.84
Traces Won	36 / 105	69 / 105
Mean Delta	+0.039 (MPP better)	
Best Single Trace	web_7 (-1.35)	int_1 (+1.34)

The left side of the delta curve tells an interesting story. Bullseye’s biggest wins are on **web_7** (-1.35 MPKI) and **int_16** (-1.14 MPKI). These are traces with a small number of hard-to-predict branches that Bullseye’s H2P identification mechanism successfully isolates and hands off to dedicated perceptrons. MPP’s broad multi-perspective approach does not capture these branches as effectively because it spreads its hardware budget across all branches rather than dedicating resources to the worst offenders.

On the right side, MPP pulls ahead on the integer-heavy traces like **int_1** (+1.34 MPKI) and **int_2** (+1.24 MPKI). These are traces with many moderately hard branches spread across the workload rather than a few extreme outliers. MPP’s MODHIST filtering, BLURRYPATH and the other novel history features work well here because the problem is history pollution rather than a small number of genuinely hard branches. Bullseye’s HIT mechanism flags some of these branches but its perceptrons cannot find a better pattern than what MPP’s richer history representation already captures.

The shape of the curve is also worth noting. The left tail drops steeply and quickly, meaning Bullseye’s wins are concentrated on a small number of traces with large improvements. The right side rises slowly and gradually, meaning MPP’s wins are spread across many traces with smaller individual improvements. This reflects the core design difference between the two predictors. Bullseye is targeted and deep, MPP is broad and consistent.

Overall MPP edges out Bullseye with a lower average MPKI of 3.4495 vs 3.4889, but the margin is small enough that the two predictors are genuinely competitive. The choice between them would depend on the workload. For workloads with a few dominant hard branches, Bullseye is likely better. For workloads with many moderately hard branches spread across the code, MPP is the stronger choice.

7.2 BALL vs Bullseye Predictor

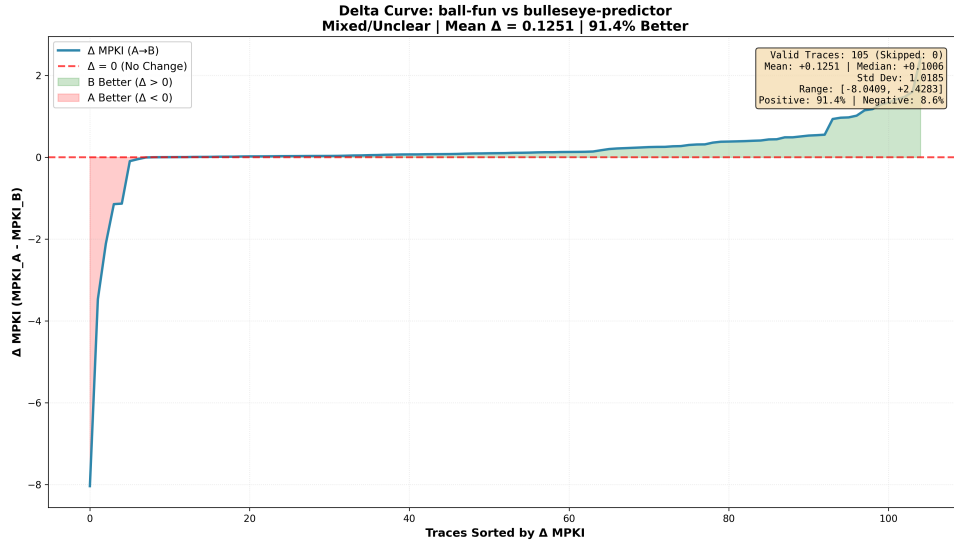


Figure 6: Delta Curve: BALL vs Bullseye Predictor

The delta curve has a clear structure: a sharp drop on the left over a few traces, a long flat middle section, and a gradual rise on the right where Bullseye performs better. Bullseye wins on 91.4

Table 9 summarises the key numbers.

Table 9: Head to Head Summary: BALL vs Bullseye

Metric	BALL	Bullseye
Avg MPKI	3.6140	3.4889
Avg Miss Rate (%)	2.8350	2.7395
CycWPPKI	158.13	154.64
Traces Won	9 / 105	96 / 105
Mean Delta	+0.125 (Bullseye better)	
Best Single Trace	int_21 (−8.04)	int_16 (+2.43)

The left side shows BALL’s wins, and they are large. On `int_21` BALL beats Bullseye by 8.04 MPKI, on `fp_13` by 3.47 MPKI, and on `fp_8` by 2.11 MPKI. These are load-data-dependent branches. Bullseye treats them as H2P and applies perceptrons, but these branches do not follow stable history patterns. Their behavior depends on runtime data values, which BALL directly models. The flat middle section shows traces where both predictors behave similarly. In these cases, both fall back to TAGE-SC-L. BALL does not activate because there are no load-dependent chains, and Bullseye does not trigger strong H2P behavior. The right side shows Bullseye’s advantage. On `int_16` it gains 2.43 MPKI and on `int_17` 1.60 MPKI. These branches depend on complex history patterns. Bullseye’s perceptrons can learn these patterns once identified as H2P. BALL has no mechanism for this case.

Overall, Bullseye performs better on average (3.4889 vs 3.6140 MPKI) and across most traces. BALL is significantly better only on the specific cases it targets. The predictors solve different problems, which is reflected in the curve.

8 Complementarity Analysis

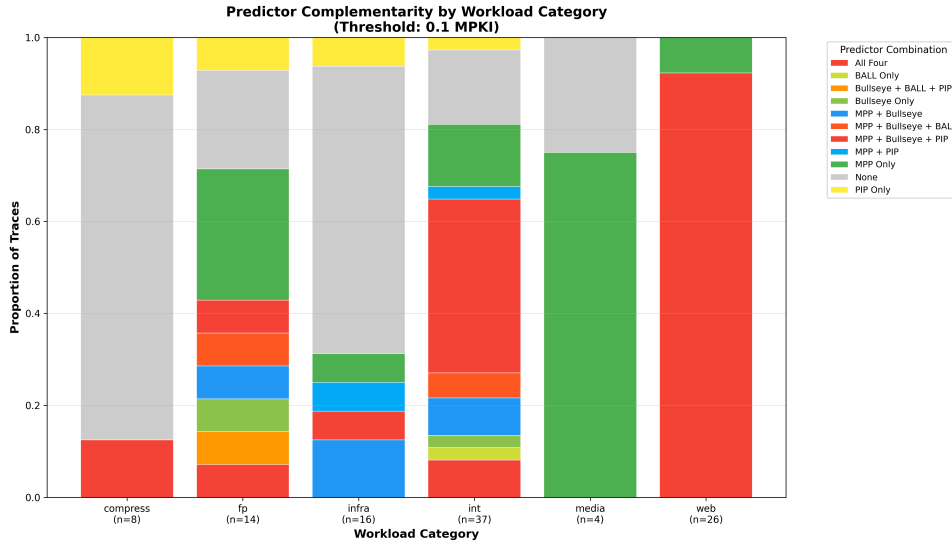


Figure 7: Predictor Complementarity by Workload Category

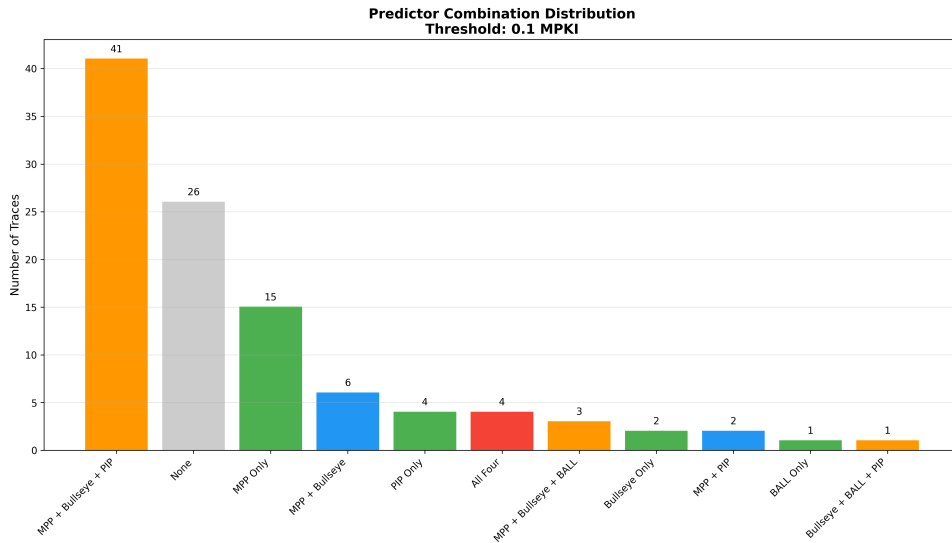


Figure 8: Predictor Combination Distribution Across All Traces

Looking at each predictor against the baseline individually only tells part of the story. A more interesting question is whether these predictors are all solving the same problem or different ones. If they overlap heavily, combining them would not help much. If they target different branches, a combined predictor could be much better than any single one. To answer this, I categorized each trace by which combination of predictors meaningfully improved it, using a threshold of 0.1 MPKI reduction over the baseline.

The most important finding is that 41 out of 105 traces fall under MPP + Bullseye + PIP together. These three predictors are largely solving the same problem through different mechanisms, so combining all three would not give triple the improvement on these traces.

BALL is a completely different story. It only appears in 9 traces total, and these are almost entirely separate from the traces that MPP, Bullseye and PIP improve. This confirms that BALL targets load-data-dependent branches which are a genuinely different problem. A combination

of BALL with any of the other three would give close to additive improvement because they are not competing for the same traces.

The 26 traces under “None” are also significant. These are traces that no predictor could improve meaningfully, representing branches that are resistant to both history-based prediction and load value prediction. This sets a practical floor on how much any of these designs can improve over the baseline and directly motivates the oracle analysis in the next section.

Looking at the category breakdown, web traces are almost entirely covered by MPP + Bullseye + PIP together, confirming that web workloads respond well to history-based approaches in general. Integer traces show more variety, with a large portion improved by all three but also a significant number that only MPP can fix, suggesting MPP’s novel history features are finding patterns that Bullseye and PIP miss. FP traces have a lot of grey meaning many fp workloads resist all predictors, but the few that BALL uniquely fixes are concentrated here. Compress remains almost entirely grey, consistent with everything seen so far.